

1. **Scoping styles:** Why global CSS collides in a component app · One CSS file per component, and BEM names · CSS Modules: class names scoped to one file · Conditional classes: the state picks the class · Inline styles: `style={{ }}` for values computed at runtime
2. **Approaches, themes and layout:** Utility-first CSS: Tailwind · CSS-in-JS: styles written in JavaScript · Choosing one styling approach · Design tokens and themes with CSS custom properties · A responsive component: breakpoints in its own CSS · Hover, focus and disabled: interaction states
3. **Tests and tooling:** Why automated tests · The testing pyramid: unit, component, end-to-end · Vitest: describe, it, expect · Tooling cheat-sheet: ESLint, Prettier, React DevTools
4. **Testing components:** Testing Library: test what the user sees · Finding elements: by role, label and text · `getBy`, `queryBy`, `findBy`: present, absent, later · Simulating the user: `userEvent` and mock callbacks · Testing async UI: waiting and mocking fetch · What not to test: implementation details · A first component test, end to end

Scoping styles

Why global CSS collides in a component app

In a component app every CSS file you import ends up in **one global stylesheet**. A class called `.title` in one component's file also styles every other `.title` on the page, and the cascade decides which rule wins.

- Imported CSS is **global**: the import says "load this", not "only for this component".
- Same specificity → the **later** rule wins, for every element that matches.
- The fix is **unique names**, not heavier selectors.

```
function TaskCard() {
  return (
    <article className="card">
      <h2 className="title">Buy milk</h2>
      <p>Due today</p>
    </article>
  );
}

function ProfileCard() {
  return (
    <article className="card">
      <h2 className="title">Ana Ruiz</h2>
      <p>Admin</p>
    </article>
  );
}

export default function App() {
  return (
    <main>
      <TaskCard />
      <ProfileCard />
    </main>
  );
}
```

Example. In the box, the author of `TaskCard` wrote an orange title, yet both titles are blue and uppercase. Both `.title` rules score (0, 1, 0), so the one later in the stylesheet wins, and it applies to **every** `.title`, not just the profile card. Delete the second rule: now both titles are orange. Rename the classes to `task-card__title` and `profile-card__title` (in the JSX and the CSS) and each card keeps its own look, whatever the order.

Common mistake. Fixing a collision by making the selector heavier: `.task-list .card .title`, or `!important`. It wins today and starts a specificity war tomorrow, when someone needs to override it. A collision is a **naming** problem: fix the name, not the weight.

One CSS file per component, and BEM names

The simplest discipline: each component gets its own CSS file next to it, and every class in it starts with the component's name, following a convention such as **BEM (Block, Element, Modifier)**: `.task-card`, `.task-card__title`, `.task-card--done`.

```
src/
  components/
    TaskCard.jsx
    TaskCard.css    /* only .task-card... classes */
    ProfileCard.jsx
    ProfileCard.css  /* only .profile-card... classes */
    index.css        /* reset, fonts, tokens: imported once in main.jsx */
```

```
import { useState } from 'react';
import './TaskCard.css';

function TaskCard({ title }) {
  const [done, setDone] = useState(false);
  return (
    <article className={done ? 'task-card task-card--done' : 'task-card'}>
      <h2 className="task-card__title">{title}</h2>
      <button className="task-card__button" onClick={() => setDone(!done)}>
        {done ? 'Undo' : 'Done'}
      </button>
    </article>
  );
}

export default function App() {
  return (
    <>
      <TaskCard title="Buy milk" />
      <TaskCard title="Call Ana" />
    </>
  );
}
```

Example. In the box, press **Done** on the first card. Only that `<article>` gets the modifier class `task-card--done`; the CSS for the modifier strikes through the title inside it. The second card is untouched because the modifier is on its own block. The class names tell you, in the DevTools Elements panel, exactly which component and which part you are looking at.

Common mistake. Believing the import scopes the CSS: "these rules are in `TaskCard.css`, so they only affect `TaskCard`". Once loaded, they are global, and they stay on the page even when no `TaskCard` is rendered. A rule like `.task-card h2 { ... }` is safe only because of the `task-card` prefix; a bare `h2 { ... }` in that file restyles every heading in the app.

CSS Modules: class names scoped to one file

A **CSS Module** is a CSS file named `*.module.css`. When a component imports it, the build tool renames every class to a unique name and hands you an object that maps your names to the real ones: `styles.title` → `"_title_1hy3p_5"`.

```
import { useState } from 'react';
import styles from './TaskCard.module.css';

function TaskCard({ title }) {
  const [done, setDone] = useState(false);
  const cardClass = done ? [styles.card, styles.done].join(' ') : styles.card;
  return (
    <article className={cardClass}>
      <h2 className={styles.title}>{title}</h2>
      <button className={styles.button} onClick={() => setDone(!done)}>
        {done ? 'Undo' : 'Done'}
      </button>
    </article>
  );
}
```

```

    </button>
  </article>
);
}

export default function App() {
  return (
    <>
      <TaskCard title="Write the report" />
      <TaskCard title="Book the dentist" />
    </>
  );
}

```

Example. In a Vite app, open the Elements panel after rendering this card: you see `<h2 class="_title_1hy3p_5">`, not `title`. Another component can declare its own `.title` in its own module and both keep their colours, in any import order. In the box, press **Done** and watch the class list of the `<article>` become `card done`.

Common mistake. Writing the class as a string after importing the module: `className="title"`. The real class is `_title_1hy3p_5`, so the string matches no rule and the styles silently vanish (no error). Always read the name from the object, `className={styles.title}`. A misspelled property, `styles.titel`, is undefined, and fails just as silently.

Conditional classes: the state picks the class

To change how a component looks when its state changes, compute the `className` from the state: add a class when a condition is true. The CSS describes each look; React only switches between them.

```

import { useState } from 'react';
import styles from './Task.module.css';

function Task({ title, urgent }) {
  const [done, setDone] = useState(false);
  const className = [styles.task, done && styles.done, urgent && styles.urgent]
    .filter(Boolean)
    .join(' ');
  return (
    <li className={className}>
      <span>{title}</span>
      <button aria-pressed={done} onClick={() => setDone(!done)}>Done</button>
    </li>
  );
}

export default function App() {
  return (
    <ul className={styles.list}>
      <Task title="Pay the rent" urgent />
      <Task title="Water the plants" />
    </ul>
  );
}

```

Example. In the box, "Pay the rent" starts with `task urgent` (red border). Press its **Done**: the class list becomes `task done urgent` and the button gets `aria-pressed="true"`, which the CSS uses to fill it in. The same attribute tells a screen-reader user that the button is now pressed.

Common mistake. Gluing a condition into a template literal, such as `task ${done && 'done'}` between backticks. When `done` is false the result is `"task false"` (and `"task undefined"` for a missing prop): a junk class that looks harmless until someone names a class `false`. Use a ternary that returns `' '`, or `filter(Boolean)`.

Inline styles: `style={{ }}` for values computed at runtime

The `style` prop takes a JavaScript object with camelCase properties, such as `style={{ width: '40%' }}`. It is the right tool for values that come from data (a width, a position, a colour the user picked); for everything else, classes

are better.

```
import { useState } from 'react';

export default function App() {
  const [done, setDone] = useState(3);
  const total = 8;
  const percent = Math.round((done / total) * 100) + '%';
  return (
    <section>
      <label>
        Tasks done: {done} of {total}{' '}
        <input type="range" min="0" max={total} value={done}
          onChange={(e) => setDone(Number(e.target.value))} />
      </label>
      <p>1. Inline width:</p>
      <div className="track">
        <div className="fill" style={{ width: percent }} />
      </div>
      <p>2. A custom property, the rule stays in CSS:</p>
      <div className="track meter" style={{ '--progress': percent }} />
    </section>
  );
}
```

Example. Move the slider. Both bars follow the data, but only the second one animates (the transition is in CSS) and changes colour on hover (:hover cannot be written inline). The component passes **one number**; the look stays in the stylesheet.

Common mistake. Writing the style as an HTML string: style="width: 40%". React throws "The style prop expects a mapping from style properties to values, not a string". The prop is an object: style={{ width: '40%' }}, with the outer braces opening JavaScript and the inner ones creating the object.

Approaches, themes and layout

Utility-first CSS: Tailwind

In **utility-first CSS** you do not write a class per component. You combine many tiny classes that each set one property, directly in the markup: className="flex items-center gap-2 p-4 rounded-lg". **Tailwind CSS is the best-known framework of this kind.**

Common Tailwind utilities and the CSS they stand for

Class	CSS
flex · grid · hidden	display: flex · display: grid · display: none
items-center · justify-between	align-items: center · justify-content: space-between
gap-2 · p-4 · px-3 · py-1 · mt-2	gap: .5rem · padding: 1rem · padding left/right .75rem, top/bottom .25rem · margin-top: .5rem
text-sm · font-bold · text-white	font-size: .875rem · font-weight: 700 · color: white
bg-blue-600 · rounded-lg · shadow	a blue background from the palette · border-radius: .5rem · a box shadow
w-full · max-w-md	width: 100% · max-width: 28rem
hover:... · focus-visible:... · md:... · dark:...	the class applies on hover · on keyboard focus · from 48rem wide · in dark mode

```
// With Tailwind installed: no CSS file for this component
function TaskCard({ title, done, onToggle }) {
  return (
    <article className="flex items-center justify-between gap-2 rounded-lg border p-4">
      <h2 className={done ? 'text-gray-500 line-through' : 'font-bold'}>{title}</h2>
      <button
        onClick={onToggle}
        className="rounded bg-blue-600 px-3 py-1 text-white hover:bg-blue-700 focus-visible:outline-2"
      >
        {done ? 'Undo' : 'Done'}
      </button>
    </article>
  );
}
```

```

    </button>
  </article>
);
}

```

```

// A few utilities written by hand (see the CSS pane): Tailwind generates
// thousands of them, but each one is this simple.
function TaskCard({ title }) {
  return (
    <article className="flex items-center justify-between gap-2 p-4 rounded-lg border">
      <h2 className="text-sm font-bold">{title}</h2>
      <button className="px-3 py-1 rounded bg-blue-600 text-white">Done</button>
    </article>
  );
}

export default function App() {
  return <TaskCard title="Buy milk" />;
}

```

Example. In the box, change `p-4` to `p-1` in the JSX: the card shrinks, without touching the CSS. That is the workflow with utilities: you edit the markup, choosing from a scale, and never write a new rule for one component.

Common mistake. Building class names at runtime, such as the template literal `bg-${color}-600`. Tailwind finds classes by scanning the **source text** for complete names; `bg-red-600` never appears whole in the file, so it is never generated and the style silently does not exist. Write the full names and pick one: `{ red: 'bg-red-600', blue: 'bg-blue-600' }[color]`.

CSS-in-JS: styles written in JavaScript

CSS-in-JS libraries such as **styled-components** let you write real CSS inside JavaScript and get back a component with those styles attached. The library generates a unique class name and inserts the CSS into the page, usually while the app runs.

```

import styled from 'styled-components';

// A component with its CSS. $primary is a "transient" prop:
// the $ keeps it from being passed on to the <button> element.
const Button = styled.button`
  padding: 6px 14px;
  border-radius: 6px;
  border: 2px solid navy;
  background: ${(props) => (props.$primary ? 'navy' : 'white')};
  color: ${(props) => (props.$primary ? 'white' : 'navy')};

  &:hover { opacity: 0.85; }
`;

export function Actions() {
  return (
    <>
      <Button $primary>Save</Button>
      <Button>Cancel</Button>
    </>
  );
}

```

Example. Rendering `Actions` produces two `<button>` elements with two different generated classes (one for each value of `$primary`) and one `<style>` tag the library added to the `<head>`. Nothing in the project is called `.button`, so nothing can collide with it.

Common mistake. Defining a styled component **inside** another component's body. Every render then creates a new component type, so React throws away the old `<button>` and mounts a new one (it loses focus and any state below it), and the library generates fresh class names each time. Define styled components at the top level of the module, like any component.

Choosing one styling approach

Plain CSS with a naming convention, CSS Modules, utility classes and CSS-in-JS all solve the same problem, scoping. Pick **one** for a project, add one global file for the reset and the design tokens, and keep inline styles for values computed at runtime.

Styling approaches compared

Approach	Scoping	Dynamic styles	Runtime cost	Good fit
Plain CSS + BEM	by convention (discipline)	classes, attributes, custom properties	none	small apps; teams fluent in CSS
CSS Modules	automatic, per file	the same, plus composes	none (build time)	the smallest step up from plain CSS
Utility classes (Tailwind)	no names to collide	conditional class strings, variants	none (build time)	teams that want one design scale
Runtime CSS-in-JS (styled-components)	automatic	props inside the CSS	yes: work in the browser	existing codebases that use it
Inline <code>style={{ }}</code>	one element	any value	small	values that come from data, only

Example. A task manager styled with CSS Modules: `index.css` holds the reset, the tokens (`--color-primary`, `--space`) and the focus ring; every component has its own `x.module.css` using `var(--...)`; the progress bar passes its percentage through `style={{ '--progress': ... }}`. A newcomer finds any style in at most two places.

Common mistake. Mixing approaches: half the components in Tailwind, some in CSS Modules, a few in styled-components, colours repeated as hexes in each. Every change starts with "how is this one styled?", and the same blue drifts into five slightly different blues. If you must migrate, do it component by component, with shared tokens.

Design tokens and themes with CSS custom properties

A **design token** is a named design decision: the primary colour, the spacing step, the corner radius. In CSS you store tokens as **custom properties** (`--color-primary: #1a1f6c`) and use them with `var(--color-primary)`. A theme, such as dark mode, is the same names with different values.

```
import { useEffect, useState } from 'react';

function ThemeToggle() {
  const [theme, setTheme] = useState('light');
  useEffect(() => {
    document.documentElement.dataset.theme = theme;
  }, [theme]);
  return (
    <button className="button" aria-pressed={theme === 'dark'}
      onClick={() => setTheme(theme === 'dark' ? 'light' : 'dark')}>
      Dark theme
    </button>
  );
}

export default function App() {
  return (
    <main className="card">
      <h1 className="card__title">Tasks</h1>
      <p>Every colour and size on this card is a token.</p>
      <ThemeToggle />
    </main>
  );
}
```

Example. In the box, press **Dark theme**. The effect sets `data-theme="dark"` on `<html>`, the second `:root` rule redefines five tokens, and every rule that uses them repaints: no component rule was duplicated. Change `--radius` to `0` in the first block: the card and the button both become square.

Common mistake. Writing dark mode as a second copy of every component rule: `.dark .card { background: ... }`, `.dark .button { ... }`, and so on for each component. The copies drift apart and every new component needs two sets of rules. Redefine the **tokens** once per theme; components keep using `var(--...)`.

A responsive component: breakpoints in its own CSS

A component can carry its own breakpoint: a media query (or a **container query) in the component's CSS changes its layout above a width, mobile-first. The rules are those of plain CSS; what changes is where they live.**

```
const tasks = ['Buy milk', 'Call Ana', 'Write the report', 'Book the dentist'];

function TaskGrid() {
  return (
    <section className="task-grid" aria-label="Tasks">
      <ul className="task-grid__list">
        {tasks.map((t) => <li key={t} className="task-grid__item">{t}</li>)}
      </ul>
    </section>
  );
}

// Drag the bottom-right corner of the dashed box to resize it.
export default function App() {
  return (
    <div className="resizable">
      <TaskGrid />
    </div>
  );
}
```

Example. In the box, drag the corner of the dashed box: below 320px the tasks stack in one column, from 320px they form two, from 560px four. The window did not change size, only the container did; a media query could not have reacted to that.

Common mistake. Giving a component a fixed width, `.task-grid { width: 800px; }`, and testing only on a laptop. On a 375px phone it overflows and the page scrolls sideways. Let components fill the space they get (`max-width` instead of `width`) and add columns with a breakpoint.

Hover, focus and disabled: interaction states

Every interactive component needs visible styles for its states: `:hover` (pointer over it), `:focus-visible` (focused from the keyboard), `:disabled`, and "this one is selected", best expressed with attributes such as `[aria-pressed="true"]` or `[aria-current="page"]`.

```
import { useState } from 'react';

const pages = ['Tasks', 'Calendar', 'Settings'];

export default function App() {
  const [page, setPage] = useState('Tasks');
  return (
    <nav aria-label="Main" className="nav">
      {pages.map((p) => (
        <a key={p} href={`#${p.toLowerCase()}`} className="nav__link"
          aria-current={p === page ? 'page' : undefined}
          onClick={(e) => { e.preventDefault(); setPage(p); }}>
          {p}
        </a>
      ))}
      <button className="nav__button" disabled>Log out</button>
    </nav>
  );
}
```

```

    </nav>
  );
}

```

Example. In the box, click inside the preview and press Tab: each link shows the orange ring, but clicking with the mouse does not. Choose **Calendar**: React moves `aria-current="page"` to it, the CSS fills it in, and a screen reader announces "current page". The disabled **Log out** is skipped by Tab and looks unavailable.

Common mistake. Removing focus rings because they look "ugly": `*:focus { outline: none; }`. Keyboard users can no longer see where they are, which fails accessibility guidelines. Style `:focus-visible` the way you want instead; mouse users will not see it.

Tests and tooling

Why automated tests

An **automated test** is code that runs your code and checks the result, so that everything can be checked again in seconds after every change. Tests catch **regressions** (something that used to work breaks) and give you the confidence to **refactor**.

```

// tasks.js
export function remaining(tasks) {
  return tasks.filter((t) => !t.done).length;
}

// tasks.test.js
import { describe, it, expect } from 'vitest';
import { remaining } from './tasks';

describe('remaining', () => {
  it('counts the tasks that are not done', () => {
    const tasks = [{ done: true }, { done: false }, { done: false }];
    expect(remaining(tasks)).toBe(2);
  });

  it('is 0 for an empty list', () => {
    expect(remaining([])).toBe(0);
  });
});

```

Example. Weeks later someone "simplifies" `remaining` and gets the filter backwards: `tasks.filter((t) => t.done)`. In watch mode the first test turns red at once, expected 1 to be 2, before the change ever reaches a user. Without the test, the counter on the screen would just be wrong until someone noticed.

Common mistake. Trusting a test you have never seen fail. A test with no `expect`, or one that checks a value the test itself set, passes whatever the code does. Make each new test fail once on purpose (break the code or the expected value) to prove it checks something; the test runner challenges do this for you with buggy components.

The testing pyramid: unit, component, end-to-end

Tests come in sizes. **Unit tests** check one function on its own; **component** (or integration) tests render a component with its children and interact with it; **end-to-end (E2E)** tests drive the whole app in a real browser against a real server. The pyramid says: many small, fast tests at the bottom, a few broad, slow ones at the top.

Three levels of tests for a React app

Level	What it covers	Tools	Speed	When it fails, you learn
Unit	one pure function: <code>validateTask</code> , <code>formatDue</code> , a reducer	Vitest	milliseconds	exactly which function is wrong
Component	a component and its children in jsdom, user events, a mocked network	Vitest + React Testing Library	tens of milliseconds	which behaviour of which component broke
End-to-end	the real app in a real browser, with the real API	Playwright, Cypress	seconds per test	that a user flow broke, somewhere

Example. For a task app: a **unit** test checks that `validateTask({ title: '' })` returns "Title is required"; a **component** test checks that `TodoForm` shows that message and does not call `onAdd`; an **E2E** test logs in, creates a task, reloads the page and sees the task still in the list.

Common mistake. Testing everything end-to-end because "it is closest to reality". The suite takes twenty minutes, fails now and then for timing reasons, and a red result only says "something in this flow broke". People stop running it. Put each check at the lowest level that can catch the bug.

Vitest: describe, it, expect

Vitest is a test runner made for Vite projects: it finds the `*.test.js` and `*.test.jsx` files, runs them and reports each test as passed or failed. A file groups tests with `describe`, defines each one with `it` (or `test`) and checks values with `expect(value).matcher(expected)`.

```
import { describe, it, expect, vi } from 'vitest';
import { addTask } from './tasks';

describe('addTask', () => {
  it('returns a new array with the task at the end', () => {
    const before = [{ id: 1, title: 'Buy milk' }];
    const after = addTask(before, { id: 2, title: 'Call Ana' });
    expect(after).toHaveLength(2);
    expect(after[1]).toEqual({ id: 2, title: 'Call Ana' }); // contents
    expect(after).not.toBe(before); // a new array
  });

  it('calls the logger once', () => {
    const log = vi.fn();
    addTask([], { id: 1, title: 'x' }, log);
    expect(log).toHaveBeenCalledTimes(1);
  });
});
```

Example. `npx vitest` prints ✓ `src/tasks.test.js` (2 tests). Change `toEqual` to `toBe` in the first test and it fails with `expected { id: 2, title: 'Call Ana' } to be { id: 2, title: 'Call Ana' } // Object.is equality`: the two objects look the same but are **different objects**, so they are not `Object.is`-equal.

Common mistake. Forgetting that a test of asynchronous code must wait: `it('loads', () => { loadTasks().then((t) => expect(t).toHaveLength(5)); })`. The test function returns before the promise settles, so it passes even when `loadTasks` is broken. Make the test `async` and `await` the call (or return the promise).

Tooling cheat-sheet: ESLint, Prettier, React DevTools

Three tools catch problems before tests do: **ESLint** finds bugs and risky patterns in the code (with the **react-hooks** plugin, broken hook rules), **Prettier** formats the code automatically, and **React DevTools** shows the component tree, props and state in the browser.

Tooling cheat-sheet

Tool	Catches	Run it	Tip
ESLint + react-hooks	bugs, unused code, broken hook rules, missing effect dependencies	<code>npm run lint</code> (eslint .); editor extension	fix warnings; do not silence exhaustive-deps
Prettier	inconsistent formatting	<code>npx prettier --write .</code> ; format on save	add <code>eslint-config-prettier</code> so ESLint does not fight it
React DevTools	wrong props or state, unexpected re-renders	browser extension: Components and Profiler tabs	select a component, then edit its state live
Elements tab	which classes and styles really apply	built into the browser (F12)	check hashed CSS Module names and the cascade
Vitest	regressions in behaviour	<code>npx vitest</code> (watch), <code>npx vitest run</code> (once)	run it in watch mode while you work

Example. ESLint underlines an effect: "React Hook useEffect has a missing dependency: 'taskId'. Either include it or remove the dependency array." Adding `taskId` to the array fixes a real bug: without it, the effect kept loading the first task after the user opened another one.

Common mistake. Silencing a warning instead of understanding it: `// eslint-disable-next-line react-hooks/exhaustive-deps` above the dependency array. The warning disappears, the stale-value bug it was pointing at stays. Fix the effect (move the function inside it, add the dependency, or use an updater function).

Testing components

Testing Library: test what the user sees

React Testing Library renders a component into a simulated page and gives you ways to find things the way a person does: by role, label and text. Its guiding principle: the more your tests resemble the way your software is used, the more confidence they can give you.

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import Counter from './Counter';

test('the count goes up when the user clicks Increment', async () => {
  const user = userEvent.setup();
  render(<Counter />); // Arrange

  await user.click(screen.getByRole('button', { name: 'Increment' })); // Act

  expect(screen.getByText('Count: 1')).toBeInTheDocument(); // Assert
});
```

Example. Read the test aloud: "render the counter; click the button named Increment; the text Count: 1 is on the page". It mentions nothing a user could not see. If `Counter` is rewritten with `useReducer`, split into two components and moved to CSS Modules, the test stays green as long as the screen behaves the same.

Common mistake. Looking for the state: wanting to check that `count` is 1, or reaching for a library that exposes component internals. State is a means; the behaviour is what the screen shows. Assert on the text, the attributes and the callbacks, and the test keeps working when the state is reorganised.

Finding elements: by role, label and text

Queries find elements in the rendered page. Prefer, in this order: `getByRole` (with name), `getByLabelText` for form fields, `getByPlaceholderText`, `getByText` for non-interactive text, `getByDisplayValue`, `getByAltText` / `getByTitle`, and only as a last resort `getByTestId`.

Implicit roles of common elements

Element	Role	Accessible name from
<code><button></code> , <code><input type="submit"></code>	button	its text, or value
<code><a href></code>	link	its text
<code><h1>...<h6></code>	heading (level 1–6)	its text
<code><input></code> (text, email), <code><textarea></code>	textbox	its <code><label></code> (or <code>aria-label</code>)
<code><input type="checkbox"></code> / "radio"	checkbox / radio	its <code><label></code>
<code><select></code>	combobox	its <code><label></code>
<code></code> , <code></code> / <code></code>	list / listitem	<code>aria-label</code> (optional)
<code></code>	img	alt
<code><nav></code> , <code><main></code> , <code><form></code>	navigation, main, form	<code>aria-label</code> (optional)
<code>role="alert"</code> / <code>role="status"</code>	alert / status	its text

Example. For a task form: `screen.getByLabelText('Task title')` finds the input through its `<label>`; `screen.getByRole('button', { name: 'Add task' })` finds the submit button; after an empty submit, `screen.getByRole('alert')` finds the error message. None of them depends on a class, an id or the order of the elements.

Common mistake. Switching to `getByPlaceholderText` or a test id because `getByRole('textbox', { name: 'Title' })` fails. The failure is telling you something: the input has only a placeholder and **no label**, so a screen reader announces it as "edit text" with no name. Fix the markup with a `<label htmlFor>`, and the role query passes.

getBy, queryBy, findBy: present, absent, later

Every query comes in three flavours. `getBy...` **returns the element or throws (it must be there now)**; `queryBy...` **returns null when there is none (to check that something is absent)**; `findBy...` **returns a promise that resolves when the element appears (for things that show up later)**. The `...AllBy...` versions return arrays.

Which query, for which question

Query	0 matches	1 match	More than 1	Waits?	Use it for
<code>getBy...</code>	throws	the element	throws	no	it is there now
<code>queryBy...</code>	null	the element	throws	no	it is not there
<code>findBy...</code>	rejects after 1 s	the element	rejects	yes: await	it will appear
<code>getAllBy...</code>	throws	[el]	[el, el...]	no	several are there now
<code>queryAllBy...</code>	[]	[el]	[el, el...]	no	counting, possibly zero
<code>findAllBy...</code>	rejects after 1 s	[el]	[el, el...]	yes: await	several will appear

Example. A form with validation: `before submitting, expect(screen.queryByRole('alert')).not.toBeInTheDocument();` after an empty submit, `expect(screen.getByRole('alert')).toHaveTextContent('Title is required')`. A list that loads: `expect(await screen.findByText('Buy milk')).toBeInTheDocument(), then expect(screen.getAllByRole('listitem')).toHaveLength(5).`

Common mistake. Checking absence with `getBy`: `expect(screen.getByText('Loading...')).not.toBeInTheDocument().` `getBy` throws **before** `expect` runs, so the test fails with "Unable to find an element" precisely when the loading message is correctly gone. Absence needs `queryBy`.

Simulating the user: userEvent and mock callbacks

`@testing-library/user-event` **simulates complete interactions**: `await user.click(button), await user.type(input, 'Buy milk')`, **the keyboard, select boxes. Combine it with `vi.fn()` to check what a component tells its parent through callback props.**

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { vi } from 'vitest';
import TodoForm from './TodoForm';

test('adds a trimmed task and clears the field', async () => {
  const user = userEvent.setup();
  const onAdd = vi.fn();
  render(<TodoForm onAdd={onAdd} />);

  const field = screen.getByLabelText('Task title');
  await user.type(field, ' Buy milk ');
  await user.click(screen.getByRole('button', { name: 'Add task' }));

  expect(onAdd).toHaveBeenCalledWith('Buy milk');
  expect(field).toHaveValue('');
});
```

Example. The test types into the field found by its label, clicks the button found by its role and name, and checks the two visible effects of a successful submit: the parent received the trimmed title, and the field is empty again. If the form forgot to trim, the failure reads expected "spy" to be called with arguments: `[ 'Buy milk' ]` and lists the actual call, `[ ' Buy milk ' ]`.

Common mistake. Forgetting `await`: `user.click(button); expect(onAdd).toHaveBeenCalled();`. The click is still in progress when the assertion runs, so the test fails, or worse, passes by luck and fails later on a slower machine. Every `user....` call returns a promise: await it (the ESLint rule `testing-library/await-async-events` reminds you).

Testing async UI: waiting and mocking fetch

A component that loads data first shows "Loading..." and the data later. Its tests must wait (`await screen.findBy...`) **and must control the network: replace fetch with a mock that answers what each test needs (data, an empty list, a 500, a network failure).**

```
import { render, screen } from '@testing-library/react';
import { afterEach, vi } from 'vitest';
import TaskList from './TaskList';

afterEach(() => vi.restoreAllMocks());

test('shows the tasks once they arrive', async () => {
  vi.spyOn(globalThis, 'fetch').mockResolvedValue(
    Response.json([
      { id: 1, title: 'Buy milk', done: false }
    ])
  );
  render(<TaskList />);
  expect(screen.getByRole('status')).toHaveTextContent('Loading');
  expect(await screen.findByText('Buy milk')).toBeInTheDocument();
});

test('tells the user when the server fails', async () => {
  vi.spyOn(globalThis, 'fetch').mockResolvedValue(new Response(null, { status: 500 }));
  render(<TaskList />);
  expect(await screen.findByRole('alert')).toHaveTextContent('Could not load tasks');
});
```

Example. The first test checks both moments: right after render the status says "Loading" (a plain `getBy`, it is there now), and `findByText` then waits for the mocked answer to be rendered. Change `findByText` to `getByText` and the test fails, because the promise from `fetch` has not resolved yet when the line runs.

Common mistake. Waiting with a fixed sleep: `await new Promise((r) => setTimeout(r, 2000))`. Every test now takes two seconds, and it still fails on a slow machine where the data needs 2.1. `findBy...` and `waitFor` wait **exactly** as long as needed, up to a timeout, and report what was on the page if it never came.

What not to test: implementation details

An implementation detail is anything the user cannot see or use: state variable names, internal functions, CSS class names, the structure of the markup. Tests that check them break when you refactor (false alarms) and pass when the user-facing behaviour breaks (false confidence).

From brittle to robust

Brittle	Robust
<code>container.querySelector('.btn-primary')</code>	<code>screen.getByRole('button', { name: 'Save' })</code>
<code>getByTestId('title-input')</code>	<code>getByLabelText('Task title')</code>
<code>expect(p.textContent).toBe('Count: 1')</code>	<code>expect(screen.getByText('Count: 1')).toBeInTheDocument()</code>
<code>expect(button.className).toContain('toggle-on')</code>	<code>expect(button).toHaveAttribute('aria-pressed', 'true')</code>
<code>expect(link).toHaveClass('active')</code>	<code>expect(link).toHaveAttribute('aria-current', 'page')</code>
a snapshot of the whole form	two or three assertions about what the test title promises

Example. The toggle story in code: the brittle test `expect(container.firstChild.className).toBe('toggle toggle-on')` breaks when the styles move to a CSS Module (the class becomes `toggle on`) and still passes when `aria-pressed` is stuck at `false`. The robust one, `expect(screen.getByRole('button', { name: 'Mute', pressed: true })).toBeInTheDocument()`, survives the refactor and catches the bug.

Common mistake. "More assertions make a better test": checking every class, every attribute and the exact markup of a component. Each one is a reason for the test to fail on a harmless change, and none adds confidence about behaviour. Assert what the test's name promises, and stop.

A first component test, end to end

From a Vite project to a green test: install Vitest, jsdom and the Testing Library packages, tell Vitest to use jsdom, load the jest-dom matchers once, write `TodoForm.test.jsx` next to the component, run `npx vitest`, and read the first failure.

```
// vite.config.js
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',
    globals: true,
    setupFiles: ['./src/setupTests.js'],
  },
});

// src/setupTests.js
import '@testing-library/jest-dom/vitest';

// src/components/ToDoForm.test.jsx
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import TodoForm from './ToDoForm';

describe('ToDoForm', () => {
  it('shows an error for an empty title', async () => {
    const user = userEvent.setup();
    const onAdd = vi.fn();
    render(<ToDoForm onAdd={onAdd} />);
    await user.click(screen.getByRole('button', { name: 'Add task' }));
    expect(screen.getByRole('alert')).toHaveTextContent('Title is required');
    expect(onAdd).not.toHaveBeenCalled();
  });
});
```

Example. `npm test` prints ✓ `src/components/ToDoForm.test.jsx` (1 test). Change the label query to `getByLabelText('Title')` in another test and the failure reads "Unable to find a label with the text of: Title", followed by the rendered HTML, where you can see the label actually says "Task title".

Common mistake. Forgetting `environment: 'jsdom'`. Vitest runs tests in plain Node by default, so the first render fails with "ReferenceError: document is not defined". And without the setup file, `toBeInTheDocument` fails with "Invalid Chai property": the jest-dom matchers were never loaded.